Soutenance parcours recherche

Formalisation d'un sous-langage d'eBPF à l'aide d'un assistant à la preuve

AROCENA Thomas
Encadrants : Ghilain BERGERON, Stephan MERZ, Vincent TRÉLAT

MINES nancy ARTEM
Loria

12 juin 2025

Table des matières

1. Introduction

2. Formalisation
   - 2.1 Structure des instructions eBPF
   - 2.2 Syntaxe du langage formel
   - 2.3 Sémantique opérationnelle

3. Interprétation abstraite
   - 3.1 Principe de l'interprétation abstraite
   - 3.2 Le type `UInt64`
   - 3.3 Tristates
   - 3.4 Intervalles
   - 3.5 Sémantique abstraite

4. Conclusion

Sommaire

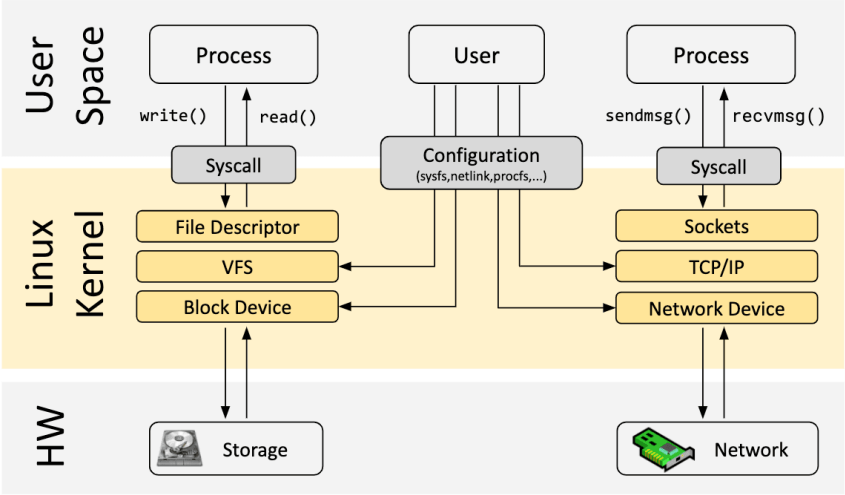
1 Introduction

2 Formalisation
- 2.1 Structure des instructions eBPF
- 2.2 Syntaxe du langage formel
- 2.3 Sémantique opérationnelle

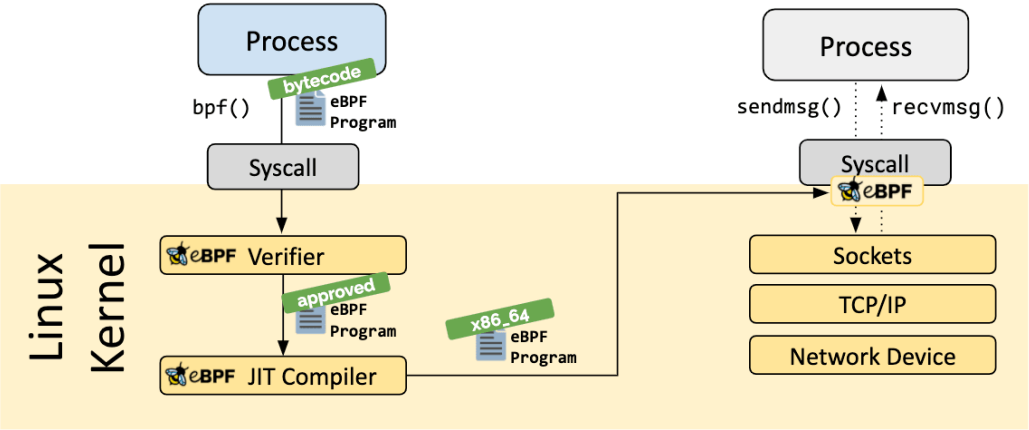
3 Interprétation abstraite
- 3.1 Principe de l'interprétation abstraite
- 3.2 Le type `UInt64`
- 3.3 Tristates
- 3.4 Intervalles
- 3.5 Sémantique abstraite

4 Conclusion

Qu’est-ce que c’est eBPF ?



Pourquoi on veut une formalisation ?



Qu'est ce que LEAN ?

Création de Type

```
inductive ℕ : Type
| zero : ℕ
| successeur (n : ℕ) : ℕ
```

Création de fonction

```
def add : ℕ → ℕ → ℕ
  | m  zero     := m
  | m  (succ n) := succ (add m n)
```

Qu'est ce que LEAN ?

Démonstration de théorème

```
theorem zero_add (n : Nat) : 0 + n = n :=
  Nat.recOn (motive := fun x => 0 + x = x)
  n
  (show 0 + 0 = 0 from rfl)
  (fun (n : Nat) (ih : 0 + n = n) =>
  show 0 + succ n = succ n from
  calc 0 + succ n
  _ = succ (0 + n) := rfl
  _ = succ n       := by rw [ih])
```

Sommaire

1 Introduction

2 Formalisation
- 2.1 Structure des instructions eBPF
- 2.2 Syntaxe du langage formel
- 2.3 Sémantique opérationnelle

3 Interprétation abstraite
- 3.1 Principe de l'interprétation abstraite
- 3.2 Le type `UInt64`
- 3.3 Tristates
- 3.4 Intervalles
- 3.5 Sémantique abstraite

4 Conclusion

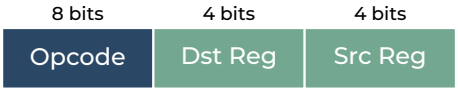
Une instruction eBPF

Une instruction eBPF

8 bits

Opcode

Une instruction eBPF



Une instruction eBPF

Une instruction eBPF

Une instruction eBPF

8 bits	4 bits	4 bits	16 bits	32 bits
Opcode	Dst Reg	Src Reg	Offset	Immediate

Par exemple : $R_0 \leftarrow R_0 + R_1$:

Une instruction eBPF

8 bits	4 bits	4 bits	16 bits	32 bits
Opcode	Dst Reg	Src Reg	Offset	Immediate

Par exemple : $R_0 \leftarrow R_0 + R_1$:

ADD|X|ALU

Une instruction eBPF

8 bits	4 bits	4 bits	16 bits	32 bits
Opcode	Dst Reg	Src Reg	Offset	Immediate

Par exemple : $R_0 \leftarrow R_0 + R_1$:

ADD|X|ALU	R_0	R_1

Une instruction eBPF

8 bits	4 bits	4 bits	16 bits	32 bits
Opcode	Dst Reg	Src Reg	Offset	Immediate

Par exemple : $R_0 \leftarrow R_0 + R_1$:

ADD|X|ALU	R_0	R_1	0

Une instruction eBPF

8 bits	4 bits	4 bits	16 bits	32 bits
Opcode	Dst Reg	Src Reg	Offset	Immediate

Par exemple : $R_0 \leftarrow R_0 + R_1$:

ADD|X|ALU	R_0	R_1	0	0

Syntaxe du langage formel

Les registres

```
inductive Register
| r0 | r1 | r2 | r3 | r4 | r5 | r6 | r7 | r8 | r9 | r10

abbrev Val := UInt64
```

Syntaxe du langage formel

Les registres

```
inductive Register
| r0 | r1 | r2 | r3 | r4 | r5 | r6 | r7 | r8 | r9 | r10

abbrev Val := UInt64
```

Les arguments

```
inductive Argument
| reg : Register → Argument
| imm : Val → Argument
```

Syntaxe du langage formel

Les instructions

```
inductive Statement
| add : Register → Argument → Statement
| jeq : Register → Argument → Int → Statement
| exit : Statement
```

Syntaxe du langage formel

Les instructions

```
inductive Statement
| add : Register → Argument → Statement
| jeq : Register → Argument → Int → Statement
| exit : Statement
```

Les programmes

```
abbrev Program : Type := List Statement
```

Sémantique opérationnelle

$$s_0 \longrightarrow s_1 \longrightarrow \cdots \longrightarrow s_n$$

Sémantique opérationnelle

$$s_0 \longrightarrow s_1 \longrightarrow \cdots \longrightarrow s_n$$

$$s = (pc, r_0, r_1, \ldots, r_{10})$$

Sémantique opérationnelle

$$s_0 \longrightarrow s_1 \longrightarrow \cdots \longrightarrow s_n$$

$s = (pc, r_0, r_1, \ldots, r_{10}) \longrightarrow$ ADD R_0, R_1

Sémantique opérationnelle

$$s_0 \longrightarrow s_1 \longrightarrow \cdots \longrightarrow s_n$$

$$s = (pc, r_0, r_1, \ldots, r_{10}) \longrightarrow \text{ADD } R_0, R_1 \longrightarrow s' = (pc + 1, r_0 + r_1, r_1, \ldots, r_{10})$$

Définition des états

Définition du type State

```
structure State where
  pc : Int
  reg : Register → Val
```

Définition des états

Définition du type State

```
structure State where
  pc : Int
  reg : Register → Val
```

- pc : compteur de programme

Définition des états

Définition du type State

```
structure State where
  pc : Int
  reg : Register → Val
```

- pc : compteur de programme
- reg : valeurs des registres

Sémantique

Fonction add

```
def add (dst : Register) (src : Argument) (s : State) : State :=
  match src with
  | Argument.imm v_src =>
      { pc := s.pc + 1,
        reg := update s.reg dst (s.reg dst + v_src) }
  | Argument.reg r_src =>
      { pc := s.pc + 1,
        reg := update s.reg dst (s.reg dst + s.reg r_src) }
```

Sémantique

Fonction add

```
def add (dst : Register) (src : Argument) (s : State) : State :=
  match src with
  | Argument.imm v_src =>
      { pc := s.pc + 1,
        reg := update s.reg dst (s.reg dst + v_src) }
  | Argument.reg r_src =>
      { pc := s.pc + 1,
        reg := update s.reg dst (s.reg dst + s.reg r_src) }
```

$s = (pc, r_0, r_1, \ldots, r_{10})$ → ADD R_0, R_1 → $s' = (pc + 1, r_0 + r_1, r_1, \ldots, r_{10})$

Exécution d'un exemple

Programme exemple

$R1 \leftarrow 10$
$R2 \leftarrow 32$
$R0 \leftarrow R0 + R1$
$R0 \leftarrow R0 + R2$
exit

Exécution d'un exemple

Programme exemple

$R1 \leftarrow 10$
$R2 \leftarrow 32$
$R0 \leftarrow R0 + R1$
$R0 \leftarrow R0 + R2$
exit

Dans notre formalisation

```
def example_program : Program :=
  [ Statement.mov Register.r1 (Argument.imm 10),
    Statement.mov Register.r2 (Argument.imm 32),
    Statement.add Register.r0 (Argument.reg r1),
    Statement.add Register.r0 (Argument.reg r2),
    Statement.exit ]
```

Exécution d'un exemple

pc	Instruction	Registres après exécution
0	Statement.mov Register.r1 (Argument.imm 10)	r1 = 10

Exécution d'un exemple

pc	Instruction	Registres après exécution
0	Statement.mov Register.r1 (Argument.imm 10)	r1 = 10
1	Statement.mov Register.r2 (Argument.imm 32)	r2 = 32

Exécution d'un exemple

pc	Instruction	Registres après exécution
0	Statement.mov Register.r1 (Argument.imm 10)	r1 = 10
1	Statement.mov Register.r2 (Argument.imm 32)	r2 = 32
2	Statement.add Register.r0 (Argument.reg r1)	r0 = 10

Exécution d'un exemple

pc	Instruction	Registres après exécution
0	Statement.mov Register.r1 (Argument.imm 10)	r1 = 10
1	Statement.mov Register.r2 (Argument.imm 32)	r2 = 32
2	Statement.add Register.r0 (Argument.reg r1)	r0 = 10
3	Statement.add Register.r0 (Argument.reg r2)	r0 = 42

Exécution d'un exemple

pc	Instruction	Registres après exécution
0	Statement.mov Register.r1 (Argument.imm 10)	r1 = 10
1	Statement.mov Register.r2 (Argument.imm 32)	r2 = 32
2	Statement.add Register.r0 (Argument.reg r1)	r0 = 10
3	Statement.add Register.r0 (Argument.reg r2)	r0 = 42
4	Statement.exit	(fin d'exécution)

Sommaire

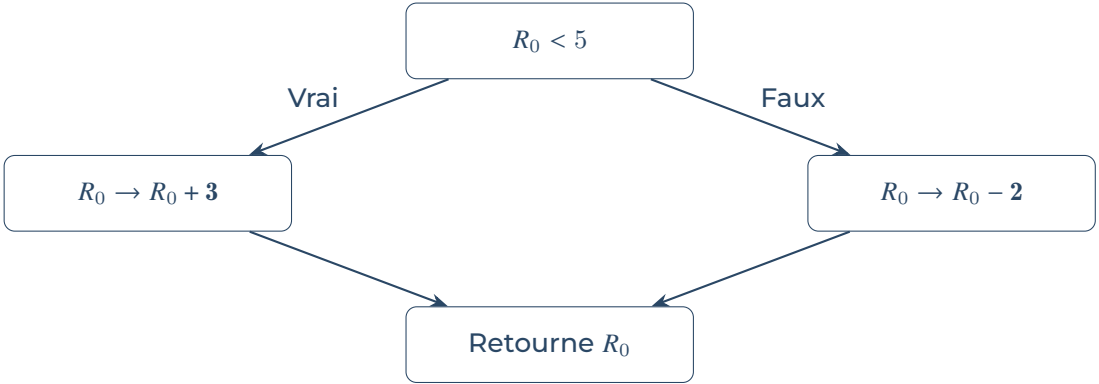
1 Introduction

2 Formalisation
- 2.1 Structure des instructions eBPF
- 2.2 Syntaxe du langage formel
- 2.3 Sémantique opérationnelle

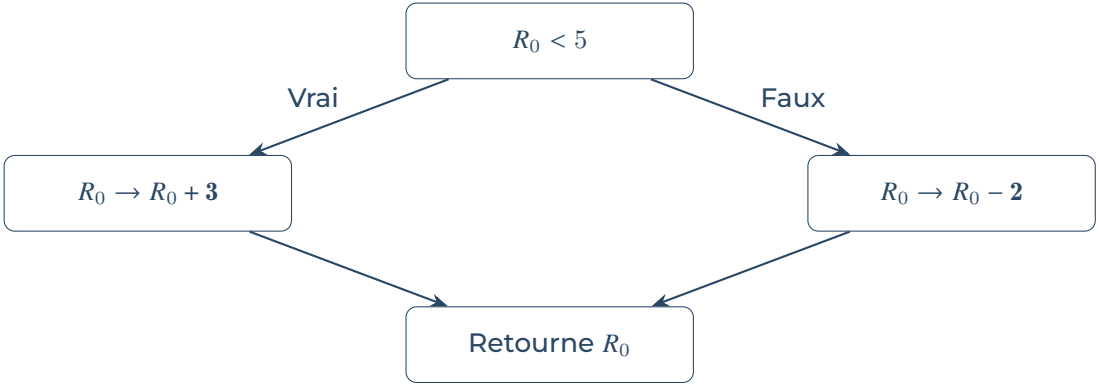
3 Interprétation abstraite
- 3.1 Principe de l'interprétation abstraite
- 3.2 Le type `UInt64`
- 3.3 Tristates
- 3.4 Intervalles
- 3.5 Sémantique abstraite

4 Conclusion

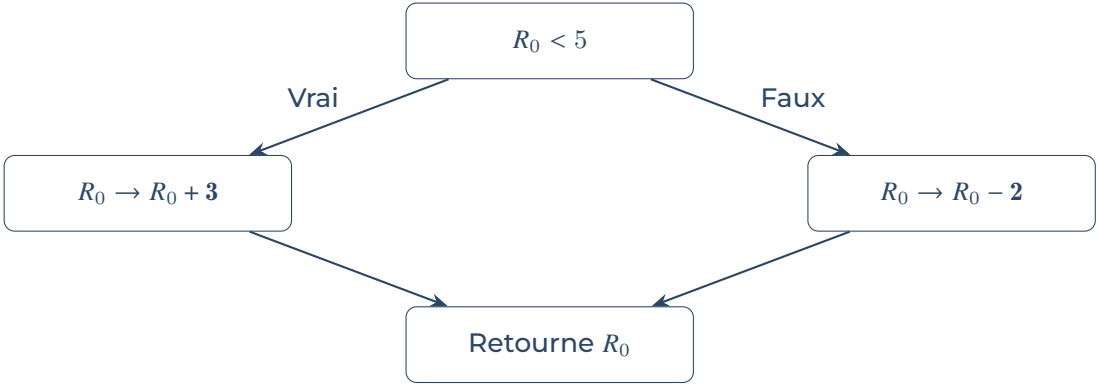
Principe de l'interprétation abstraite



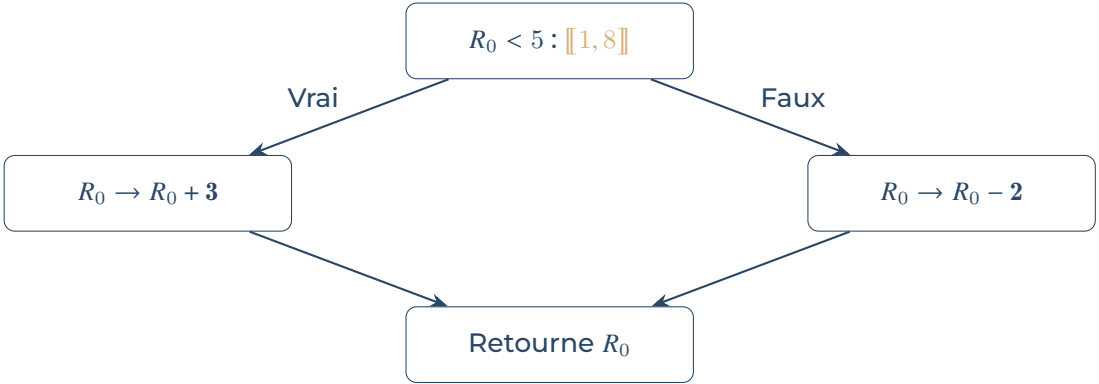
Principe de l'interprétation abstraite



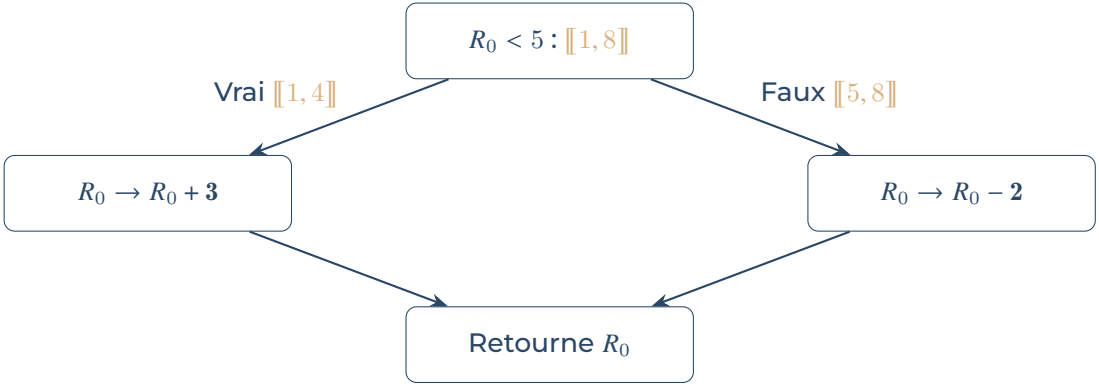
Principe de l'interprétation abstraite



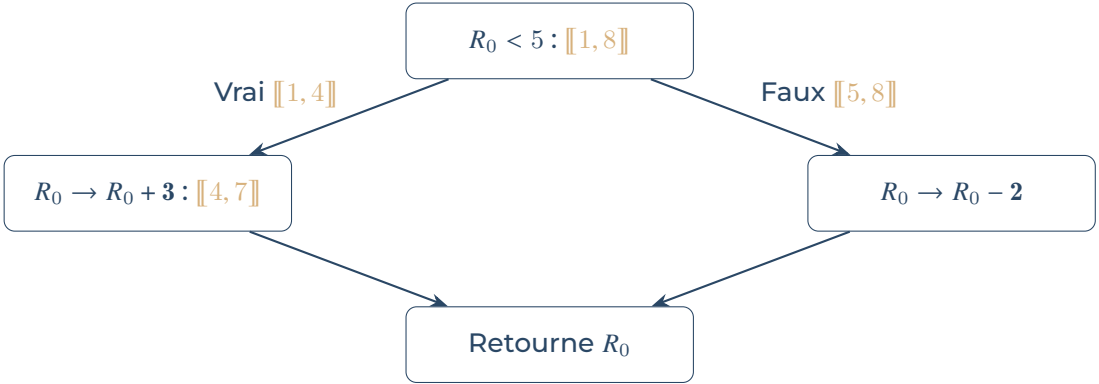
Principe de l'interprétation abstraite



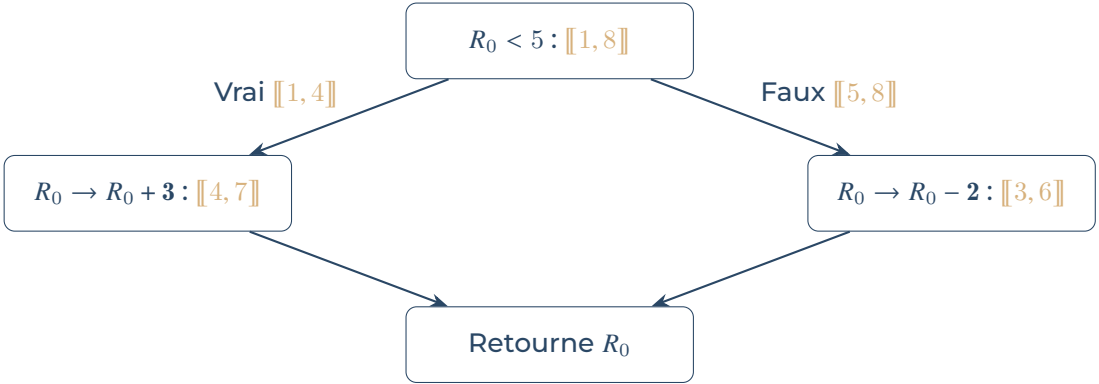
Principe de l'interprétation abstraite



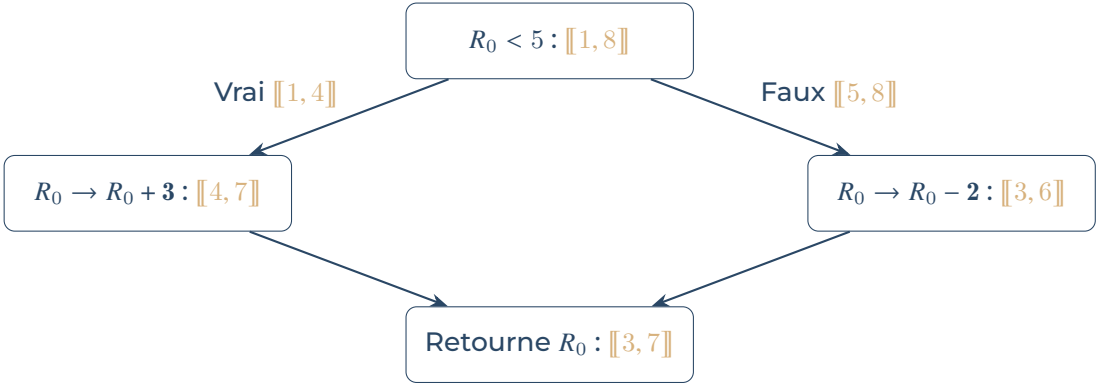
Principe de l'interprétation abstraite



Principe de l'interprétation abstraite



Principe de l'interprétation abstraite



Le Type UInt64

Comportement modulaire de UInt64

```
#eval (2^64 : Nat).toUInt64  -- = 0
```

Le Type UInt64

Comportement modulaire de UInt64

```
#eval (2^64 : Nat).toUInt64  -- = 0
```

- Représentation binaire de taille fixe (vecteur de 64 bits).

Le Type UInt64

Comportement modulaire de UInt64

```
#eval (2^64 : Nat).toUInt64  -- = 0
```

- Représentation binaire de taille fixe (vecteur de 64 bits).
- Opération binaire : AND, OR, XOR, décalages logiques.

Le Type UInt64

Comportement modulaire de UInt64

```
#eval (2^64 : Nat).toUInt64  -- = 0
```

- Représentation binaire de taille fixe (vecteur de 64 bits).
- Opération binaire : AND, OR, XOR, décalages logiques.
- Théorèmes : `0 &&& x = 0`, `x &&& y = y &&& x`

Les tristates

Notation intuitive

$$011?$$

Les tristates

Notation intuitive

$$011? = \{0110, 0111\}$$

Les tristates

Notation intuitive

$$011? = \{0110, 0111\}$$

Notation pratique

$$(0110, 0001)$$

Les tristates

Notation intuitive

$$011? = \{0110, 0111\}$$

Notation pratique

$$(0110, 0001)$$

Type Tnum

```
structure Tnum where
    v : UInt64
    m : UInt64
    wf : v &&& m = 0
```

Opération binaire sur les Tnum

$\vee$	0	1
0	0	1
1	1	1

Opération binaire sur les Tnum

$\vee$	0	1
0	0	1
1	1	1

$$\begin{array}{cc} & ? \\ \vee & 1 \\ \hline = & 1 \end{array} \qquad \begin{array}{cc} & 1 \\ \vee & ? \\ \hline = & 1 \end{array} \qquad \begin{array}{cc} & ? \\ \vee & 0 \\ \hline = & ? \end{array} \qquad \begin{array}{cc} & 0 \\ \vee & ? \\ \hline = & ? \end{array} \qquad \begin{array}{cc} & ? \\ \wedge & ? \\ \hline = & ? \end{array}$$

On considère $v = v_1 \vee v_2$, $\mu = m_1 \vee m_2$ et enfin $m = \mu \wedge \neg v$

Opération binaire sur les Tnum

or sur les Tnum

```
def or (t₁ t₂ : Tnum) : Tnum :=
  let val := t₁.v ||| t₂.v
  let μ := t₁.m ||| t₂.m
  { v := val, m := μ &&& (~~~val),
    wf := by
    have : val &&& (μ &&& ~~~val) = ~~~val &&& val &&& μ := by rw [←
     UInt64.and_assoc, UInt64.and_comm, ← UInt64.and_assoc],
    have h : ~~~val &&& val = 0 := by rw [UInt64.and_comm, UInt64.and_not_eq_zero],
    rw [this, h, UInt64.zero_and]
  }
```

Opération binaire sur les Tnum

or sur les Tnum

```
def or (t₁ t₂ : Tnum) : Tnum :=
  let val := t₁.v ||| t₂.v
  let μ := t₁.m ||| t₂.m
  { v := val, m := μ &&& (~~~val),
    wf := by
    have : val &&& (μ &&& ~~~val) = ~~~val &&& val &&& μ := by rw [←
     UInt64.and_assoc, UInt64.and_comm, ← UInt64.and_assoc],
    have h : ~~~val &&& val = 0 := by rw [UInt64.and_comm, UInt64.and_not_eq_zero],
    rw [this, h, UInt64.zero_and]
  }
```

On définit de manière analogue $\wedge$ et $\oplus$ sur les Tnum.

Addition de Tnum

$$\begin{array}{rr} & 0 \\ + & 0 \\ \hline = & 0 \end{array} \qquad \begin{array}{rr} & 1 \\ + & 0 \\ \hline = & 1 \end{array} \qquad \begin{array}{rr} & 1 \\ + & 1 \\ \hline = & 10 \end{array} \qquad \begin{array}{rr} & ? \\ + & 0 \\ \hline = & ? \end{array} \qquad \begin{array}{rr} & ? \\ + & 1 \\ \hline = & ?? \end{array} \qquad \begin{array}{rr} & ? \\ + & ? \\ \hline = & ?? \end{array}$$

Addition de Tnum

$$\begin{array}{rr} & 0 \\ + & 0 \\ \hline = & 0 \end{array} \qquad \begin{array}{rr} & 1 \\ + & 0 \\ \hline = & 1 \end{array} \qquad \begin{array}{rr} & 1 \\ + & 1 \\ \hline = & 10 \end{array} \qquad \begin{array}{rr} & ? \\ + & 0 \\ \hline = & ? \end{array} \qquad \begin{array}{rr} & ? \\ + & 1 \\ \hline = & ?? \end{array} \qquad \begin{array}{rr} & ? \\ + & ? \\ \hline = & ?? \end{array}$$

$$\begin{array}{rr} & 1?01 \\ + & 1??0 \\ \hline = & 1???1 \end{array}$$

Vérifions ce que donne le résultat par rapport à l'addition sur les entiers que l'on connait :

$$1?01 = \{9, 13\}$$

$$1??0 = \{8, 10, 12, 14\}$$

$$1???1 = \{17, 19, 21, 23, 25, 27, 29, 31\}$$

$$\{9, 13\} + \{8, 10, 12, 14\} = \{17, 19, 21, 23, 25, 27\}$$

Addition de Tnum

add sur les Tnum

```
def add (t₁ t₂ : Tnum) : Tnum :=
  let sv := t₁.v + t₂.v
  let sm := t₁.m + t₂.m
  let S := sv + sm
  let q := S ^^^ sv
  let h := q ||| t₁.m ||| t₂.m
  { v := sv &&& (~~~h), m := h,
    wf := by
    rw [UInt64.and_assoc]
    have : ~~~h &&& h = 0 := by rw [UInt64.and_comm, UInt64.and_not_eq_zero]
    rw [this, UInt64.and_zero]
  }
```

Intervalles d'entiers 64 bits

Type I

```
abbrev I : Type := Interval UInt64
```

- $\emptyset$
- $[a, b]$ avec $a \leq b$

Addition sur I

$$\begin{array}{rcccc} + & : & I \times I & \to & I \\ & & (i_1 = [a_1, b_1], i_2 = [a_2, b_2]) & \mapsto & [(a_1 + a_2), (b_1 + b_2)] \end{array}$$

Cependant, deux subtilités importantes doivent être prises en compte :

1. L'écriture $i = [a, b]$ n'a pas de sens lorsque i est vide.
2. Les additions sur `UInt64` sont en modulo 2^{64} :

$$[0, 2^{64} - 1] + [1, 2] = [1, 1]$$

car $(2^{64} - 1) + 2 \equiv 1 \mod 2^{64}$.

Addition sur I

$$i_1 + i_2 = \begin{cases} \emptyset & \text{si } i_1 = \emptyset \text{ ou } i_2 = \emptyset, \\ [0, 2^{64} - 1] & \text{si } a_1 + a_2 \geq 2^{64} \text{ ou } b_1 + b_2 \geq 2^{64}, \\ [a_1 + a_2, b_1 + b_2] & \text{sinon.} \end{cases}$$

Addition sur I

add sur I

```
def add (i₁ i₂ : I) : I :=
  | _ , none => none
  | none , _ => none
  | some i₁, some i₂ =>
    let h := i₁.snd.toNat + i₂.snd.toNat ≤ UInt64.size - 1
    let j := i₁.fst.toNat + i₂.fst.toNat ≤ UInt64.size - 1
    if h && j then
      some
      { fst := i₁.fst + i₂.fst, snd := i₁.snd + i₂.snd, fst_le_snd := by admit}
    else
      some
      { fst := 0, snd := (UInt64.size - 1).toUInt64, fst_le_snd := by admit }
```

Opérations binaire sur I

or sur I

```
def or (t : Tnum) (i₁ i₂ : I) : I :=
  | _ , none => none
  | none , _ => none
  | some i₁, some i₂ =>
     some { fst := i₁.fst ||| i₂.fst, snd := t.v ||| t.m, fst_le_snd := by admit}
```

Opérations binaire sur I

or sur I

```
def or (t : Tnum) (i₁ i₂ : I) : I :=
  | _ , none => none
  | none , _ => none
  | some i₁, some i₂ =>
     some { fst := i₁.fst ||| i₂.fst, snd := t.v ||| t.m, fst_le_snd := by admit}
```

On définit de manière analogue $\wedge$ et $\oplus$ sur I.

Ensemble des états par pc

Ensemble des états

On associe à chaque position pc un couple d'ensembles d'états abstraits. Pour un entier $k \in [\![1, pc]\!]$, on note

$$\Sigma_k = (\Sigma_k^*, \overline{\Sigma_k}) \quad \text{où tous les états } s \in \Sigma_k^* \cup \overline{\Sigma_k} \text{ vérifient } s.pc = k$$

Ensemble des états par pc

Ensemble des états

On associe à chaque position pc un couple d'ensembles d'états abstraits. Pour un entier $k \in [\![1, pc]\!]$, on note

$$\Sigma_k = (\Sigma_k^*, \overline{\Sigma_k}) \quad \text{où tous les états } s \in \Sigma_k^* \cup \overline{\Sigma_k} \text{ vérifient } s.pc = k$$

File de travail

$$F := (u_1, u_2, \ldots, u_k) \quad \text{avec } u_1 \leq u_2 \leq \cdots \leq u_k$$

Idée de l'algorithme

Initialisation

$$\Sigma_1 = (\emptyset, \{S_{init}\}),\ F = (1)$$

Idée de l'algorithme

Initialisation

$$\Sigma_1 = (\emptyset, \{S_{init}\}),\ \ F = (1)$$

Nouveaux états depuis une instruction $P[u_1]$

$$\forall j \in [1, n], \quad I_{u_1}^{j} = \left\{ s \in P[u_1](\overline{\Sigma_{u_1}}) \mid s.pc = j \right\}$$

Idée de l'algorithme

Initialisation

$$\Sigma_1 = (\emptyset, \{S_{init}\}),\ F = (1)$$

Nouveaux états depuis une instruction $P[u_1]$

$$\forall j \in [1, n], \quad I_{u_1}^j = \left\{ s \in P[u_1](\overline{\Sigma_{u_1}}) \mid s.pc = j \right\}$$

Mise à jour des ensembles

$$\Sigma'_j = (\Sigma_j^*, \overline{\Sigma_j} \cup I_{u_1}^j) \quad \text{pour } j \neq u_1, \quad \Sigma'_{u_1} = (\Sigma_{u_1}^* \cup \overline{\Sigma_{u_1}}, I_{u_1}^{u_1})$$

On ajoute à F les j tels que $I_{u_1}^j \neq \emptyset$

Sommaire

1 Introduction

2 Formalisation
- 2.1 Structure des instructions eBPF
- 2.2 Syntaxe du langage formel
- 2.3 Sémantique opérationnelle

3 Interprétation abstraite
- 3.1 Principe de l'interprétation abstraite
- 3.2 Le type `UInt64`
- 3.3 Tristates
- 3.4 Intervalles
- 3.5 Sémantique abstraite

4 Conclusion

Travail accompli & Perspectives

Travail accompli

Travail accompli & Perspectives

Travail accompli

- Formalisation complète d’un sous-langage eBPF

Travail accompli & Perspectives

Travail accompli

- Formalisation complète d’un sous-langage eBPF
- Définition de deux domaines abstraits

Travail accompli & Perspectives

Travail accompli

- Formalisation complète d'un sous-langage eBPF
- Définition de deux domaines abstraits
- Début de la sémantique abstraite

Travail accompli & Perspectives

Travail accompli

- Formalisation complète d'un sous-langage eBPF
- Définition de deux domaines abstraits
- Début de la sémantique abstraite

Perspectives

Travail accompli & Perspectives

Travail accompli

- Formalisation complète d'un sous-langage eBPF
- Définition de deux domaines abstraits
- Début de la sémantique abstraite

Perspectives

- Finir l'implémentation du la sémantique abstraite

Travail accompli & Perspectives

Travail accompli

- Formalisation complète d'un sous-langage eBPF
- Définition de deux domaines abstraits
- Début de la sémantique abstraite

Perspectives

- Finir l'implémentation du la sémantique abstraite
- Étendre le sous-langage -> formalisation plus large de la mémoire

Travail accompli & Perspectives

Travail accompli

- Formalisation complète d'un sous-langage eBPF
- Définition de deux domaines abstraits
- Début de la sémantique abstraite

Perspectives

- Finir l'implémentation du la sémantique abstraite
- Étendre le sous-langage -> formalisation plus large de la mémoire
- Prouver des propriétés grâce à la sémantique abstraite

Retour personnel

Positif	Negatif
Apprentissage de Lean	Loin de ma formation
Découvert de la recherche	Simplification du sujet à l'extrême
Compétence en code	Prise d'initiative difficile